

显著提升了系统的可扩展性与可维护性。

2.4.2 ROS 系统框架

ROS 是 Robot Operating System 的缩写，即机器人操作系统。虽然名字中包含操作系统，但 ROS 实际上是一个面向机器人硬件场景的系统框架，可运行在 Linux 内核以及其他兼容 POSIX 接口的操作系统内核之上。作为操作系统的系统框架，ROS 提供了一系列面向机器人场景应用的系统级服务，实现了底层复杂的异构硬件资源的抽象，并提供了应用之间和应用与服务之间交互的 IPC 机制。

图 2.12 展示了 ROS 的层次架构。简而言之，ROS 可以分为基础设施层、通信层和应用层。其中，基础设施层包括 ROS 中的 Python 和 C++ 等运行环境、ROS 文件系统、ROS 包管理机制等。在基础设施层之上运行 ROS 的通信层，提供基础的 ROS IPC 机制以及 ROS 的数据分发服务（Data Distribution Service）等。ROS 的应用层是由一个个节点所构成的计算图，这些节点之间可以使用 ROS 的通信层进行直接调用，或者通过对一些话题（Topic）发布消息或订阅消息以进行间接通信。

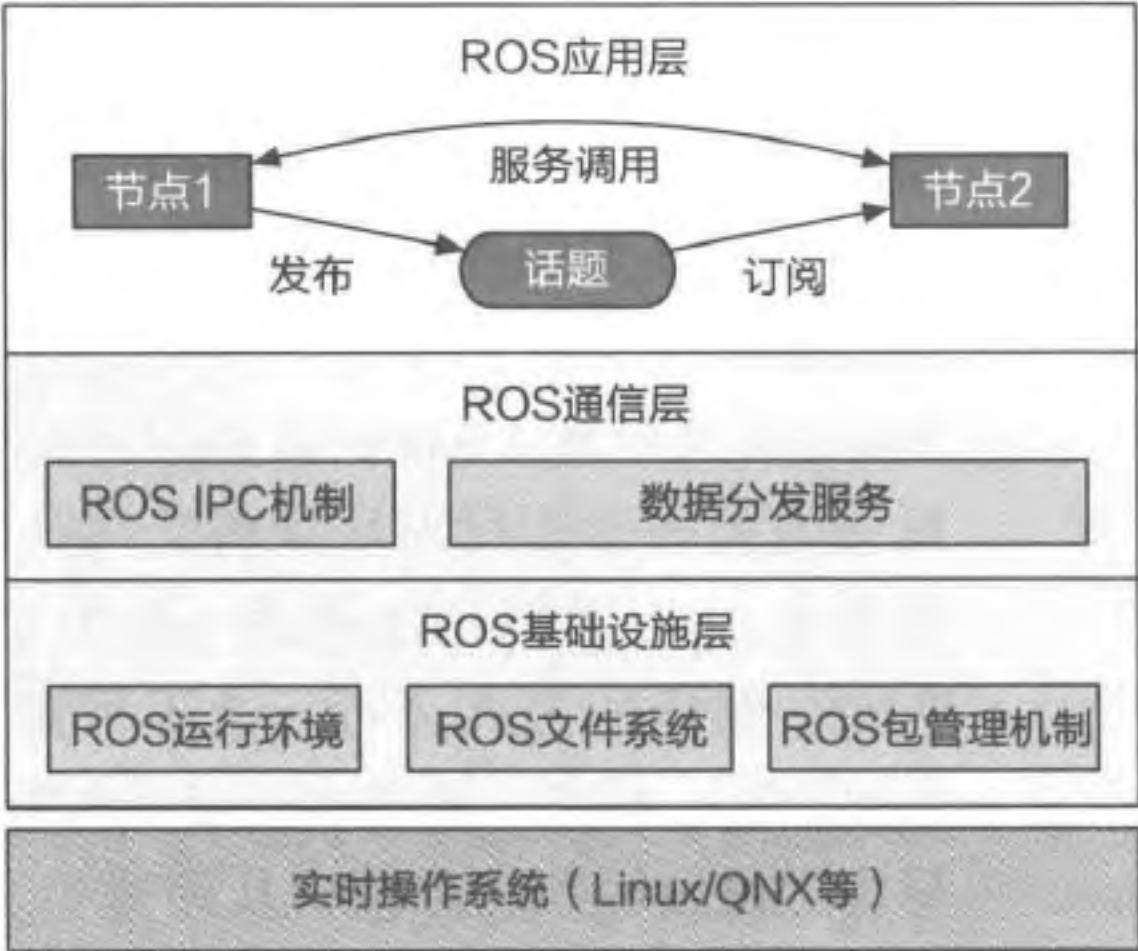


图 2.12 ROS 的层次架构

ROS 节点与计算图。ROS 采用计算图的形式来组织各种服务。每个服务构成一个节点，每个节点执行一个具体的计算处理，包括机器人中所需的感知、融合、规划与控制等。运行在 ROS 上的应用可以被表示成一个“图”的架构，不同的节点表示不同的应用（它们既可以是接收信息者，也可以是信息发布者）。ROS 上常见的应用包括：接收、发布、处理来自传感器（Sensor）的数据，以及对处理过程进行控制的节点与计划节点等。

与微内核的设计思想类似，ROS 同样将其系统框架分解为多个服务，不同的服务之间通过 ROS 提供的 IPC 进行通信。这种解耦式的架构使得 ROS 易于部署，甚至可以部署在分布式系统上。

服务与动作。ROS 每个组件的设计是以服务为中心的：服务接收一个请求，执行请求，然后回复结果。一个节点可以向另一个提供服务的节点发送请求，然后等待结果。服务的模型是一次性的同步通信，即当服务的请求和回复完成后，连接会被断开，进行下一次交互时必须再次建立连接。节点还可以提供动作（Action）。动作和服务类似，主要的区别在于动作的假设是服务端处理请求的时间会很长，并且可能需要在处理的过程中就获取一些中途的返回值。在实际的使用中，一般通过动作来让机器人执行相对复杂的任务。

话题。话题是 ROS 中比较新的处理方式，是一类单向的异步通信机制。话题机制将通信的双方称为发布者（Publisher）和订阅者（Subscriber）。发布者节点首先需要向主节点（一个特殊的全局节点）注册一个话题，之后该发布者会以消息的形式发布关于该话题的新内容。希望接收该话题的订阅者节点则通过主节点获得对应话题发布者的消息，基于这个消息，订阅者节点可以直接连接到发布者节点来接收最新的话题消息。在话题模式下，发布者和订阅者是多对多的关系，即单个节点可以成为多个话题的发布者，也可以成为多个话题的订阅者。

数据分发服务。ROS 2.0 引入了数据分发服务（Data Distribution Service, DDS）以支持实时性。从架构的角度，DDS 可以被看作一个系统中间件，处于 ROS 应用层和 ROS 基础设施层之间。DDS 提供了上面介绍的 ROS 基本通信接口和语义，如话题方式的发布者和订阅者模型或服务类型的 IPC 等。之所以将 DDS 分离出来，是为了加速对性能有重要影响的通信。不同的硬件厂商可以针对自己的机器人环境实现 DDS 的中间件，只要最终这些 DDS 的实现能够满足对应的协议接口即可。这些 DDS 是基于特定硬件实现的，因此可以进行可定制的优化。这样一来，ROS 的应用只需要考虑 DDS 的抽象，而不需要考虑底层硬件的细节，便可以使用当前环境下最高效的通信方式。

有了 DDS 抽象后，ROS 2.0 可以抛弃 ROS 中基于主节点的服务发现方式。这依赖于 DDS 提供的一套分布式发现系统（Distributed Discovery System），从而避免在主节点上出现性能瓶颈。ROS 需要通过 DDS 提供的 API 来获取，如当前存活的节点、当前的话题、当前注册的动作等信息。对于这些信息的查询都被 ROS 进行了抽象，对 ROS 上的应用而言其实是透明的。

DDS 内部可以利用共享内存的方式来传递本地数据。在 ROS 1.0 中，从系统底层来看，一次传输消息是先序列化数据，然后通过网络（TCP）把数据传递出去，即使是本地数据交互也会通过 localhost 等接口传输。而在 ROS 2.0 中，ROS 将数据交接给 DDS 层。具体的 DDS 实现可以使用共享内存等方式在不同节点之间共享数据，从而极大地提升传输性能。

计算图架构与 ROS IPC。ROS 系统框架的设计将计算任务组件化与服务化，形成独立的 ROS 节点，各个节点通过 ROS IPC（如话题、服务、动作等方式）进行通信。DDS 的引入能够在支持多种通信方式的情况下，尽可能地优化底层通信性能。这些设计充分体现了机制与策略分离的原则。类微内核设计的解耦式架构，使得 ROS 系统框架及其应用易于部署在硬件环境差异极大的机器人平台上，并且显著提升了单个节点的可复用能力。

2.5 操作系统设计：Worse is better?

小背景：PC loser-ing 问题

PC loser-ing 问题最早出现在 MIT ITS 操作系统的一个场景中^[15]；

- 进程 A 发起了一个系统调用，该系统调用在内核态运行的时间很长，例如从磁盘读取一个很大的文件，此时进程 A 处于等待状态。
- 进程 B 给进程 A 发送一个信号 (Signal)，进程 A 需要立即从内核态返回用户态处理这个信号。

在这个场景中，进程 A 被打断的系统调用应该怎么办呢？ITS 与 UNIX 做出了不一样的选择。ITS 的处理方式是：①内核保存当前 I/O 的状态（例如已经读到的磁盘位置与读取的内容）与应用上下文；②内核更新进程 A 的上下文以反映系统调用已经完成的部分工作；③从内核返回进程 A 并处理信号；④进程 A 使用更新后的应用上下文继续执行，并重新发起系统调用；⑤内核处理系统调用，从上次被中断时的位置继续执行（如从上次读取的位置继续读磁盘）。而 UNIX 的处理方式则是操作系统直接返回一个错误号，表示系统调用出错；至于“判断系统调用是否正确，如果出错该如何处理”的责任，则丢给了应用程序。

可以看到，两个操作系统的原则是不同的：ITS 是通过较为复杂的操作系统设计和实现，使系统调用的接口对于应用程序来说是简单的（接口简单，实现复杂）；而 UNIX 则避免了复杂的操作系统实现，但应用程序必须考虑调用接口可能出现的更多情况，并自己处理这些情况（实现简单，接口复杂）。

1989 年，Richard Gabriel 在一篇论文中抛出了著名的操作系统设计观点 “Worse is better”^[16]：在所有的设计考量中，简单性最重要，其中实现简单比接口简单更重要；为了简单性，必要时甚至可以牺牲部分正确性、一致性与功能完备性。他对比了两种设计方法：一种是 MIT 方法，以设计了 Common Lisp 以及 Common Lisp Object System (CLOS) 的 MIT 为代表；另一种是新泽西方法[⊖]，以设计了 UNIX 与 C 语言系统的贝尔实验室为代表。他认为，基于 Worse is better 理念的新泽西方法对于设计软件尤其是操作系统来说更为合适。

表 2.3 展现了两种设计方法在理念上的差异。MIT 方法认为：

- 设计要简单，接口简单比实现简单更重要，也就是要“将复杂留给自己，简单留给用户”。
- 在所有可观测到的方面，设计一定要正确，不正确的设计是不能容忍的。
- 设计要确保一致性，为了一致性，设计者甚至可以牺牲部分简单性与完备性。

表 2.3 MIT 方法与新泽西方法的对比

	MIT 方法	新泽西方法
简单性	接口简单比实现简单更重要	实现简单比接口简单更重要
正确性	需要 100% 正确	简单比正确更重要
一致性	必须满足一致性	可以为了简单性而牺牲一致性
完备性	必须尽可能完备	可以为了简单性而牺牲完备性

⊖ 贝尔实验室在美国新泽西州。

- 设计要尽可能地覆盖现实中各方面的重要场景，所有合理的、可预期的场景必须要覆盖到，完备性的优先级高于简单性的优先级。

而新泽西方法则认为：

- 设计要简单，但实现简单比接口简单更重要，简单原则高于其他原则。
- 在所有可观测到的方面，设计一定要正确，但为了设计简单，可以略微牺牲正确性。
- 设计要尽可能保持一致，但为了设计简单，可以牺牲一致性。例如，如果为了一致性而需要考虑一种不常见的情况，导致实现变得更复杂，那么可以不考虑这种情况。
- 设计要尽可能覆盖现实中各方面的重要场景，所有合理的、可预期的场景必须要覆盖到；但为了简单性，完备性可以让步。在保持设计简单性的前提下，可以牺牲一致性（尤其是接口一致性）来实现完备性。

现实中，很多操作系统的设计在一开始采用的是 Worse is better 的新泽西方法：先设计出覆盖基本情况、具备基本功能的操作系统来占领市场，然后根据新的需求不断迭代和完善功能，从而向 MIT 方法逼近。例如，本章提到的 Linux 0.01 版只有 8 102 行代码，而 Linux 5.7 内核则达到了 2 870 万行代码。如果在一个操作系统的起步阶段就求全责备，则要么很难坚持到实现完成，要么只能应用在某些特定的场景。本章提到的 IBM 1991 年斥资 20 亿美元设计的 Workplace 操作系统，就因为一开始设计过于复杂而最终失败。

值得一提的是，在 Richard Gabriel 的 Worse is better 的观点被广泛传播后，Nickieben Bourbaki 写了一篇名为“Worse is better is worse”的文章^[17]对这个观点进行了批判，认为 Worse is better 的设计理念——如“牺牲接口的简单性来换取实现的简单性”——是有害的，很容易误导刚入行的新人或学生，使他们以此为理由去做些简单而偷懒的设计。有意思的是，后来人们发现，Nickieben Bourbaki 实际上是 Richard Gabriel 的笔名。我们认为，不存在绝对正确的设计方法，要针对实际场景的需要做出合理的取舍。

2.6 ChCore：教学科研型微内核操作系统

ChCore 是一个面向教学和科研开发的微内核操作系统，其基本架构如图 2.13 所示。ChCore 采用了微内核架构的设计理念，将大部分的操作系统功能以系统服务的形式运行在用户态。以文件系统为例，在 Linux 或 Windows 中，文件系统通常作为操作系统内核的一部分直接运行在内核态；然而在 ChCore 中，文件系统不再属于内核的一部分，而是作为一个系统服务以独立

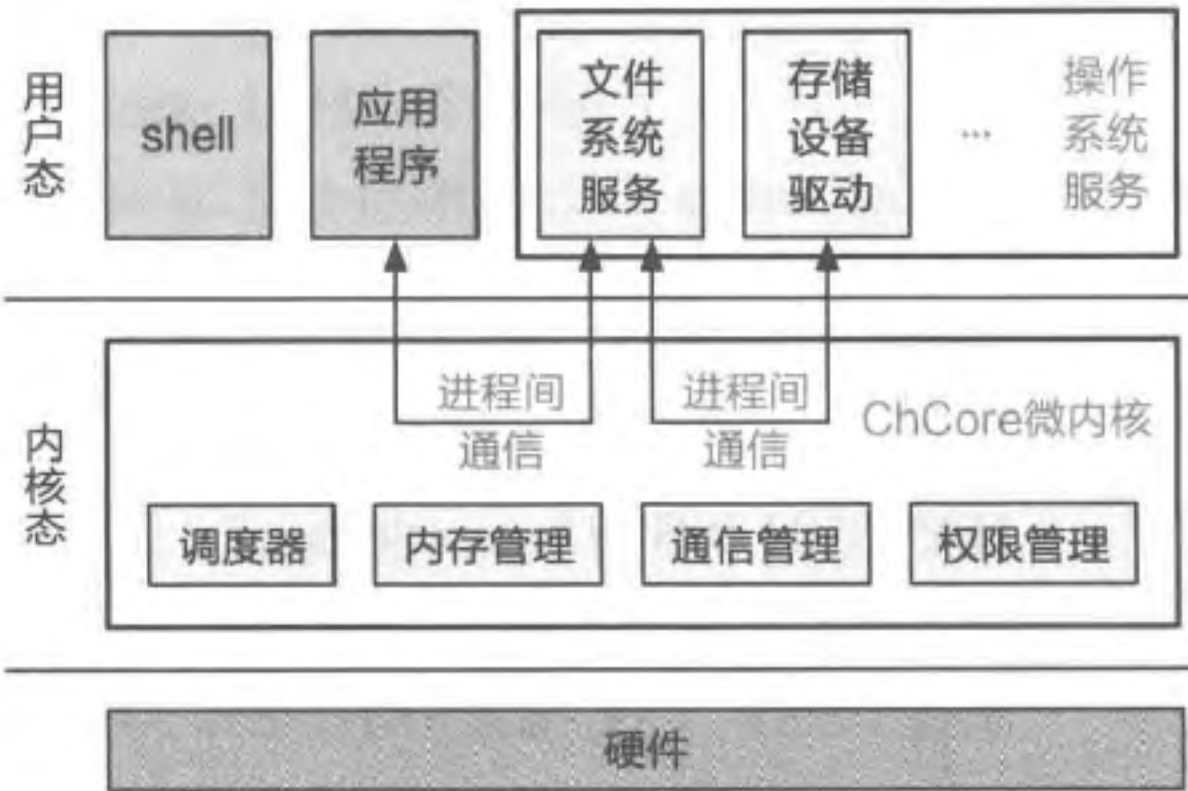


图 2.13 ChCore 系统架构示意图